Arquitectura de la Solución

Sesión 5 · Implementación de Arquitecturas Serverless para LLMs

Campo	Detalle
Curso	Diseño de Infraestructura Escalable para LLMs
Institución	BSG Institute
Autoría	Andres Rojas
Fecha	2026-11-10
Versión	0.1 (borrador técnico)
Estado	Verificación parcial: ver sección 9

1. Propósito y alcance

Este documento describe la arquitectura de referencia utilizada en la Sesión 5: un gateway de inferencia con semántica serverless (autenticación, validación, caché, concurrencia acotada, procesamiento asíncrono con cola) y su equivalente en AWS, Azure y Google Cloud. Escenario ficticio conductor: SoporteIA, asistente de soporte de una aseguradora regional.

2. Requerimientos

2.1 Funcionales

- Chat síncrono y por streaming (SSE).
- Trabajos asíncronos con Idempotency-Key y consulta de estado.
- Aislamiento por tenant.
- Métricas y estadísticas operativas.

2.2 No funcionales

Atributo	Requerimiento (hipótesis de diseño)
Latencia	SLO de TTFT interactivo a definir con negocio
Escalabilidad	Escala a cero; concurrencia máxima configurable
Resiliencia	Reintentos con backoff, DLQ, idempotencia
Seguridad	Mínimo privilegio, sin secretos de larga vida, guardrails por capas
Costo	Variable por uso; presupuesto y alertas

3. Arquitectura lógica

Flujo síncrono: Cliente → gateway → función (auth, rate limit, guards) → caché → modelo. Flujo asíncrono: Cliente → Submit (202) → cola → worker → modelo → almacén de estado; fallos → DLQ.

```
flowchart LR
  C[Cliente] --> GW[API Gateway]
  GW --> F[Funcion: auth + guards]
  F -->|hit| K[(Cache TTL por tenant)]
  F -->|miss| M[Modelo]
  F -.->|sin slot| R[429 + Retry-After]
  GW -->|POST /jobs| Q[(Cola)] --> W[Worker] --> M
  Q -. N intentos .-> D[(DLQ)]
```

4. Patrones de diseño

- Queue-Based Load Leveling y Competing Consumers.

- Idempotent Receiver (clave en borde + escritura condicional).
- Cache-Aside con single-flight.
- Bulkhead (semáforo de concurrencia) y backpressure (429).
- Retry con backoff exponencial y jitter; Dead Letter Queue.

5. Comparación multicloud

Aspecto	AWS	Azure	Google Cloud
Cómputo	Lambda	Functions (Flex Consumption)	Cloud Run / functions
Cola	SQS	Service Bus	Pub/Sub
Modelo administrado	Bedrock	Microsoft Foundry	Plataforma de agentes (Gemini)
Agentes	Bedrock Agents / AgentCore + Strands	Foundry Agent Service + Microsoft Agent Framework	ADK + runtime de agentes
Identidad	IAM roles	Managed Identity + RBAC	Service Accounts + IAM

Nota de vigencia: los nombres y el estado de servicios y frameworks cambian con frecuencia; verificar en la documentación oficial antes de cada dictado.

6. Análisis de seguridad

Riesgo	Control implementado	Límite
Inyección de prompt	InputGuard: NFKC, patrones ES/EN, política off/flag/block	Heurística evadible
Salida insegura	Sanitización de imágenes externas/HTML activo	Streaming: patrones partidos entre chunks
Fuga de datos	Redacción de PII, logs con huellas	Regex no exhaustivas
Abuso de costo	Límite de prompt/tokens, rate limit, semáforo	Rate limit en memoria: usar almacén compartido
Aislamiento	Tenant en clave de caché y jobs	—

Marcos de referencia: OWASP GenAI Security Project (Top 10 LLM y Agentic; verificar edición vigente), NIST AI RMF 1.0 y AI 600-1, ISO/IEC 42001:2023.

7. Decisiones arquitectónicas (ADR)

ADR-001 · Cola entre cliente y modelo para cargas no interactivas

Estado: aceptada. Decisión: Usar SQS/Service Bus/Pub/Sub con DLQ, idempotencia y concurrencia máxima.

Consecuencias: + resiliencia y control de costo; − latencia y complejidad de consulta de estado.

ADR-002 · Caché solo con temperature=0 y clave por tenant

Estado: aceptada. Decisión: La clave incluye tenant, modelo, parámetros y prompt normalizado; TTL obligatorio.

Consecuencias: + costo/latencia; − obsolescencia y riesgo de fuga si se omite el tenant.

ADR-003 · Mitigación de cold start por capas

Estado: aceptada. Decisión: Init fuera del handler; SnapStart/min-instances/always-ready solo si el SLO lo exige; keep_alive en inferencia autogestionada.

Consecuencias: + latencia; − costo fijo.

8. FinOps

```
costo_LLM = tokens_ent*P_ent + tokens_sal*P_sal
costo_efectivo = (1-h)*costo_LLM + h*costo_cache
N* = costo_fijo_mensual / costo_variable_por_peticion
```

Los precios no se incluyen: deben tomarse de las páginas oficiales vigentes.

9. Estado de verificación

Elemento	Estado
Gateway, 51 pruebas, ruff, mypy	Verificado (proveedor simulado)
Experimentos E1–E3	Ejecutados con simulación
Frontend (build)	Verificado; prueba manual pendiente
Ollama real, build Docker	Pendiente
Laboratorios AWS/Azure/GCP	Verificación estática; no ejecutados en cuentas reales